

Week 7

33_Build_vs_Image

Compose Fields

- Build
- Image

Features

- Custom image creation
- Existing image usage

Simple Meaning

- Understanding whether to create an image or use an existing one.

Description

The build field in Docker Compose tells Docker to create an image using a Dockerfile located in a specified directory. The image field specifies an existing image that should be downloaded from a registry. Developers use build when they need custom application images and image when using pre-built images. Understanding the difference helps optimize deployment workflows. Build provides flexibility during development, while image improves consistency in production environments. Both fields are essential components of Docker Compose configurations.

34_Service_Dependency_Ordering

Service Dependencies

- depends_on
- Startup sequencing
- Service relationships

Features

- Controlled initialization
- Reduced startup failures
- Better reliability

Simple Meaning

- Managing the order in which containers start.

Description

Many applications depend on other services being available before they can function correctly. Docker Compose provides dependency management through the `depends_on` directive. For example, a web application may require a database service to start first. Service dependency ordering ensures that required containers are started before dependent services. This reduces initialization failures and improves application stability. Proper dependency management is especially important in complex multi-container environments.

35_Database_Backend_Frontend_Deployment

Multi-Container Applications

- Database
- Backend
- Frontend

Features

- Full-stack deployment
- Service integration
- Simplified management

Simple Meaning

- Deploying all parts of an application together.

Description

Modern web applications typically consist of three main components: a database, backend server, and frontend interface. Docker Compose allows these components to be deployed and managed together. Containers communicate through Docker networks while maintaining separation of responsibilities. This architecture improves maintainability and scalability. Developers can easily test complete application environments using a single command. Multi-container deployment is one of the most common use cases for Docker Compose.

36_WordPress_MySQL_Deployment

Use Case

- WordPress
- MySQL

Features

- Content management system
- Database integration

Simple Meaning

- Deploying a WordPress website using Docker Compose.

Description

WordPress is one of the world's most popular content management systems and relies on a database such as MySQL for storing content. Docker Compose simplifies deployment by defining both services in a single configuration file. Volumes ensure that website content and database data persist across container restarts. Networking allows WordPress to communicate with MySQL seamlessly. This setup provides a practical example of multi-container application deployment.

37_NodeJS_MongoDB_Deployment

Use Case

- Node.js
- MongoDB

Features

- Backend development
- NoSQL database support

Simple Meaning

- Running a Node.js application with MongoDB.

Description

Node.js is a popular JavaScript runtime used for backend development, while MongoDB is a widely used NoSQL database. Docker Compose enables developers to deploy both services together and establish communication between them. Environment variables can be used to configure database connections. This setup is commonly used in modern web applications and microservices. Using Docker Compose ensures consistency across development and production environments.